SpringGuard 技术报告

SpringGuard：基于 Open Code Review 的 Spring 智能审查增强

项	内容
作品名称	SpringGuard 智能审查增强
参赛方向	AI 开发工具与开源协作
作品形态	开发工具 / 二次开发增强（CLI + 规则包 + 评测 + 演示）
日期	2026 年 10 月

> 说明：正文不含校名、校徽与指导教师信息。对照 AIC·AI+开源技术报告参考大纲撰写，正文建议 ≤15 页。

一、作品概述

1.1 背景与问题

Java / Spring Boot 是校园实训与中小团队后端的主流技术栈。代码审查中反复出现、且易漏到线上的问题包括：

1. `@Transactional` 同类 `this.xxx()` **自调用**导致代理失效，事务不生效；
2. Controller `@RequestBody` **缺少** `@Valid` / `@Validated`；
3. 对 `Map.get` 等可空结果直接调用实例方法引发 **NPE**；
4. 字符串拼接或 MyBatis 注解误用 `${}` 带来注入风险；
5. 事务方法中 **吞掉异常**，破坏回滚语义。

通用大模型审查存在漏报、位置漂移与不可复核问题；仅依赖人工又成本高。需要一种**可演示、可复现、开源边界清晰**的 Spring 场景审查增强方案。

1.2 目标、方向与形态

- **目标**：无大模型时仍能稳定检出核心 Spring 缺陷；可接入 Open Code Review（OCR）与本地 Ollama 作语义补充；提供金标准评测；成果可 CLI 复用。
- **参赛方向**：**AI 开发工具与开源协作**。

- **形态**：在合法开源审查工具之上的领域增强（规则引擎 + 规则包 + 编排 + 评测 + 演示页），而非从零重写通用审查引擎。

1.3 核心亮点

1. **自研 SpringGuard**：面向 Controller / Service / Mapper 的确定性规则，结果可复现。
2. **开源底座融合**：不改上游源码，通过 `.opencodereview/rule.json` 与外部编排挂接 OCR。
3. **可验证**：`eval/gold.json` 六类缺陷，本轮 spring_guard **Recall = 1.0**。
4. **非薄封装**：规则、评测、样例、演示与清单均为团队成果，清单中清晰披露上游。

二、需求分析与总体方案

2.1 需求分析

优先级	需求	说明
P0	稳定检出 Spring 高发缺陷	演示与落地的底线
P0	可离线/无 API Key 跑通	比赛演示与校园机房友好
P1	可接 OCR + 本地模型	体现 AI 与开源融合
P1	金标准评测	满足「可验证效果」
P2	可视化结果页	降低答辩理解成本

现有做法痛点：直接部署 OCR 对 Spring 特有模式依赖模型临场发挥；纯 Sonar 类工具缺少开源 Agent 叙事与可配置领域提示。

2.2 总体方案

```
Spring 工程 / samples/spring-buggy
        │
        ├─► SpringGuard（自研静态规则）──► findings
        │
        └─► Open Code Review（上游，可选）
                 + .opencodereview/rule.json（自研规则包）
                 ──► OCR findings
        │
        └─► merge → eval（Recall）→ web 演示
```

输入：Spring 源码树。输出：归一化 findings JSON、评测结果、可选浏览器展示。

2.3 可行性与成本

- **算力**：演示以确定性扫描为主（秒级）；OCR 可选，本地小模型即可连通。
- **软件**：Python 3、可选 Node.js OCR CLI、可选 Ollama。
- **周期**：两日内完成样例、规则、评测、演示与材料草稿，符合学生团队成本。

三、AI 技术与开源及第三方资源选择

3.1 AI 技术作用

环节	作用	如何验证
OCR Agent（可选）	对代码做语义级审查与解释	`ocr llm test`；与金标准对比
Spring 规则包提示	引导 Agent 关注 Spring 专项模式	规则文件可审计
确定性 Guard	**非生成式**，保证底线召回	`eval/run_eval.py`

为何需要 AI：语义跨文件、命名变体、复杂事务边界等问题适合 Agent；**为何不能只靠 AI**：比赛与工程要求可复核，故以 Guard 为主、OCR 为增强。

3.2 开源及第三方资源选择

详见附录《开源及第三方资源使用清单》。要点：

- **Open Code Review**（Apache-2.0）：审查 CLI 底座；未改源码。
- **Ollama + qwen3.5**（可选）：本地 OpenAI 兼容接口。
- **Python 3**：自研引擎与评测。

3.3 自主开发与权利边界

归属	内容
上游	OCR 实现与默认引擎
自研	SpringGuard、规则包、流水线、金标准、样例、演示页、文档
AI 辅助	可用编程助手辅助编写；规则与金标准均经团队设计与运行复核

3.4 融合方式（非薄封装）

融合 = **上游可扩展点**（规则 JSON、CLI） + **自研领域引擎与评测闭环**。相对「只跑 OCR」：增加确定性 Spring 规则与可复现 Recall；相对「只写静态脚本」：保留挂接开源 AI 审查的能力与规则包资产。

四、设计与实现过程

4.1 核心功能与流程

1. 对目标工程运行 `spring_guard.py` → `out/spring_guard.json`
2. （可选）运行 OCR scan/review → OCR findings
3. `run_pipeline.py` 合并 → `out/merged.json`
4. `eval/run_eval.py` 对照 `gold.json` 计算 Recall
5. `demo.sh` 启动本地页展示结果

4.2 关键实现

- **文件筛选**：按路径启发式识别 Controller / Service / Mapper。
- **规则示例**：缺 `@Valid`；`Map.get` 后直接方法调用；`@Transactional` 下 `this.method(`；catch 吞异常；SQL/LIKE 拼接；注解 `${`。
- **注释剥离**：避免 BUG 注释中的 `@Valid` 造成假阴性。
- **OCR 超时策略**：演示默认 `--skip-ocr`，保证答辩可复现。

4.3 过程与 AI 工具说明

开发中可使用生成式 AI 辅助脚本与文档起草；纳入作品的代码与金标准均经本地运行与人工核对。未将模型输出直接当作「已检出缺陷」的唯一依据。

五、成果展示与效果验证

5.1 成果展示

- 仓库：`springguard/`（脚本、规则、样例、评测、web、docs）
- 一键演示：`./scripts/demo.sh` → `http://127.0.0.1:5193/`
- 口播稿：`docs/DEMO_SCRIPT.md`；视频分镜：`docs/VIDEO_SHOTLIST.md`

5.2 验证方法

- **案例验证**：人工植入 6 类缺陷的 `samples/spring-buggy`
- **基准对比**：spring_guard / ocr_only / hybrid 三种模式
- **主指标**：Recall = 命中金标准数 / 金标准总数

5.3 验证结果（脚本复现）

模式	Findings	TP	Recall	漏检
spring_guard	7	6	**1.00**	无
ocr_only（本轮未完成长时 scan）	0	0	0.00	全部
hybrid（本轮等同 Guard）	7	6	**1.00**	无

已达成：Guard 对六类缺陷全召回；演示闭环可跑。
尚未充分达成：本地小模型完整 OCR 扫描耗时长，ocr_only / 完整 hybrid 对比待更强算力或 CI 补全；Precision 待负例集。

复现命令：

```
python3 scripts/run_pipeline.py --skip-ocr
python3 eval/run_eval.py
```

六、创新性与开放价值

6.1 创新性

相对裸 OCR：将 Spring 高发模式固化为确定性规则，评测可复核。
相对通用静态分析：提供开源 Agent 挂接点与领域规则包，形成「确定性 + AI」双轨。
相对薄封装：自研引擎、评测协议与样例构成实质改进，而非换皮部署。

6.2 开放成果

拟/已开放：规则引擎源码、规则包、评测脚本与金标准说明、缺陷样例、演示页与文档。建议许可证与仓库 LICENSE 保持一致（推荐 Apache-2.0 或 MIT，提交前团队确认）。

6.3 社区贡献与复用

当前以「可复用增强包」形态供他人挂接 OCR；后续可向上游提规则示例 Issue/PR。他人复用方式：拷贝 `spring_guard.py` + `rule.json` + `eval/` 协议到自有 Spring 仓。

6.4 维护计划

扩大负例与 Precision；CI 接入；补充 `@Async`、多数据源事务等规则；完善中文文档。

七、合规、安全与风险说明

7.1 许可合规

OCR 为 Apache-2.0，未改源码、保留署名与 NOTICE 义务；模型侧以模型卡为准（清单标「待核验」项须在提交前完成）。

7.2 数据与隐私

样例为团队构造的演示代码，不含真实个人信息与生产密钥。不得将用户私有仓库扫描结果擅自公开。

7.3 AI 输出与内容安全

Agent 可能幻觉；本方案以 Guard 与金标准抑制误用。审查结果供人工确认，不自动改生产库。

7.4 知识产权与公开边界

自主部分归团队依法享有权利的主体；上游权利不变。提交材料不含校名与指导教师信息。

7.5 其他风险

规则基于启发式，存在误报/漏报；不适用于混淆严重或非 Java 工程；OCR 长时推理可能超时。

八、总结与展望

本作品完成了「开源审查底座 + Spring 确定性增强 + 可复现评测 + 可演示闭环」，验证了学生团队在短周期内做出实质二次开发的可行性。后续将补全 OCR 对比实验、Precision 与更多规则，并视情况向上游贡献规则示例。

九、附录

附录 A 开源及第三方资源使用清单

见同目录 `开源及第三方资源使用清单.md`（提交时作为技术报告附件一并打包）。

附录 B 成果与运行

- 安装与运行：仓库根目录 `README.md`
- 评测纪要：`docs/EVAL_REPORT.md`、`eval/results.json`
- 答辩 HTML 幻灯：`docs/PPT.html`（浏览器打开，←→ 翻页；可另存为 PDF）
- 视频分镜：`docs/VIDEO_SHOTLIST.md`

附录 C 作品简介（报名系统用，≤300 字）

Spring 应用中事务自调用、入参未校验、MyBatis 动态 SQL 注入等问题高发，通用大模型审查易漏报且难复核。本作品基于阿里开源 Open Code Review，增加面向 Spring 的确定性规则引擎 SpringGuard 与项目级规则包，形成「确定性

检测 + AI 深度审查」混合流水线；在含六类已知缺陷的样例上，Guard 召回率达 1.0，结果可脚本复现。相对直接部署上游工具，本方案明确自研边界，突出开源底座与团队增强的融合，适合本地演示与校园工程落地。